

Assignment 3: Machine Learning for Time Series

Frederik Appel Vardinghus-Nielsen

May 19, 2026

Todo list

1 FD001: Single Condition, Single Fault Mode

1.1 Problem understanding

The RUL task is a regression problem since the target variable is a time period. The variable in this task is in a unit of cycles, which are strictly discrete, but there are no predefined classes and the range has no upper bound, so regression is more fitting.

One row in the dataset represents one cycle, the unit of time. Each row contains info on engine value, operation settings, time and sensor values.

RUL labels must be constructed from the given failure time in cycle number.

- The train series run until failure, so the labels will be the total number of cycles minus the current cycle number.
- The test series do not run until failure but the number of cycles until failure when the series ends is given so the labels will be the length of the test series plus the RUL at the end of the series minus the current cycle number.

1.2 Exploratory data analysis

Figure 1 shows the distribution of engine lifetimes in the FD001 training set. The lifetimes range from roughly 130 to 360 cycles with most engines failing between 150 and 250 cycles.

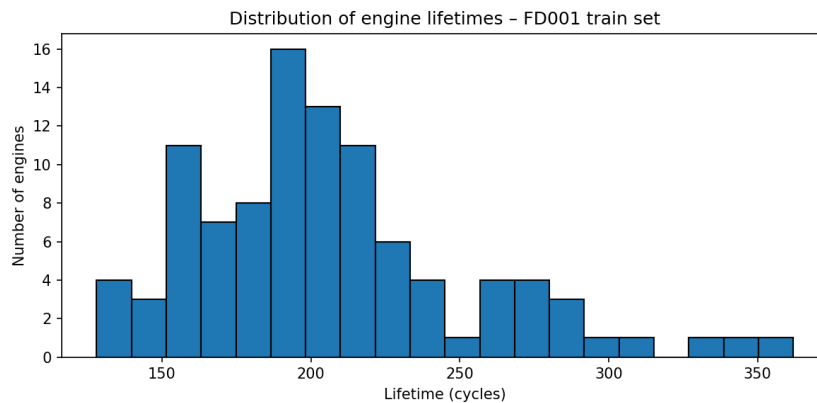


Figure 1: Distribution of engine lifetimes in the FD001 training set.

Figure 2 shows the raw sensor trajectories for a sample of engines. Many sensors are constant (e.g. `sensor_1`, `sensor_5`) and carry no degradation signal, while others (e.g. `sensor_2`, `sensor_11`) show a clear monotonic trend over the engine lifetime.

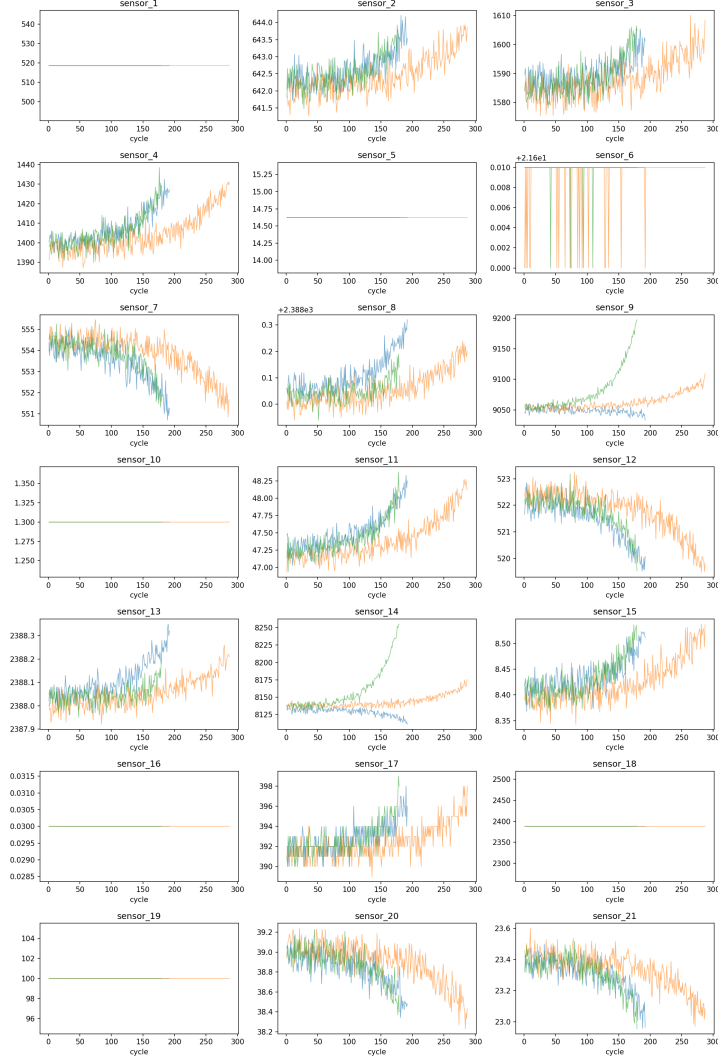


Figure 2: Per-sensor trajectories for a sample of engines in the FD001 training set.

The sensor channels were chosen based upon their Spearman correlation with the cycle number. The Spearman correlation was calculated individually for each sensor for each engine and then aggregated across engines by mean. The sensor channels with mean Spearman correlation above 0.5 and standard deviation below 0.3 were selected. This resulted in 12 sensors: `sensor_2`, `sensor_3`, `sensor_4`, `sensor_7`, `sensor_8`, `sensor_11`, `sensor_12`, `sensor_13`, `sensor_15`, `sensor_17`, `sensor_20`, and `sensor_21`.

1.3 Data preparation

- **RUL label construction:** constructed as piecewise-linear with a cap of 125 cycles. RUL beyond 125 is not expected to show sensor values that are much different from other healthy data points, so capping prevents the model from attempting to learn less relevant patterns and focus of the part where health deterioration is more evident.
- **Train/validation split:** A classic 80/20 split by engines for train/val is used.
- **Normalization:** The sensor values are normalized by subtracting the mean and dividing by the standard deviation of each sensor channel across engines in the training set. This is classic Z-score normalization, which ensures that all sensor channels are on a comparable

scale and prevents any single channel from dominating the learning process due to larger numeric values.

- **Fixed random seed:** A fixed random seed (42) for the train/validation split is used to ensure reproducibility.

2 FD002: Multi-Condition, Single Fault Mode

2.1 Problem understanding

FD002 shares the same turbofan degradation task as FD001 but contains six distinct operating conditions rather than one. Engines in FD002 switch between conditions within a single run, so a raw global normalisation would conflate sensor variation due to condition with genuine degradation signal.

2.2 Exploratory data analysis

Figure 3 shows the distribution of engine lifetimes in the FD002 training set. Lifetimes span a wider range than FD001, reflecting the additional variability introduced by multi-condition operation.

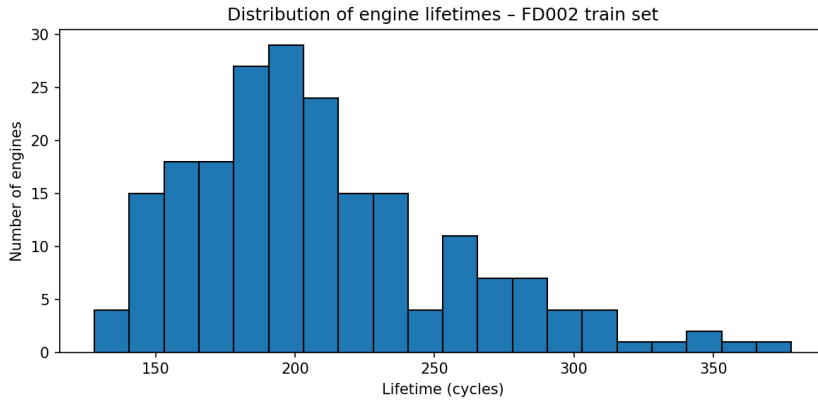


Figure 3: Distribution of engine lifetimes in the FD002 training set.

Figure 4 shows raw sensor trajectories for a sample of FD002 engines. Compared to FD001, the trajectories are noisier due to condition switching within each run, motivating the per-cluster normalisation described below.

Figure 5 shows the mean value of each operating setting per cluster. The six clusters are well separated: `op_setting_1` and `op_setting_3` each span roughly 10–15 distinct levels, while `op_setting_2` (altitude-like, range ≈ 0 –0.84) also varies meaningfully across clusters. This confirms that the operating conditions are discrete and that a cluster-specific normalisation is appropriate.

Figure 6 shows per-cluster normalised sensor trajectories for the same 12 sensors used as model inputs. After normalisation, trajectories from different operating conditions are aligned around zero, making degradation trends visible across the full engine lifetime.

2.3 Data preparation

Operating conditions are identified by clustering the three operation setting columns with KMeans ($k = 6$) on the training set. The cluster assignment is added as a column `op_condition` and used to normalise each sensor separately within each cluster (per-cluster z-score). The



Figure 4: Per-sensor trajectories for a sample of engines in the FD002 training set.

`op_condition` column is appended as an additional input feature so the models can condition their predictions on the current regime. All other preparation steps (RUL cap, train/val split, fixed random seed) are identical to FD001. The same set of 12 sensors selected for FD001 is reused for FD002.

3 Methods

3.1 Feature engineering – raw fixed-window pipeline

A window length $w = 30$ is used. This is treated as a hyperparameter in a later part of the assignment.

3.2 Feature engineering – feature-sequence pipeline

The feature sequence pipeline is based on the raw fixed-window pipeline but with additional feature engineering. For each window, the following suggested features are calculated for each sensor channel:

- Rolling mean

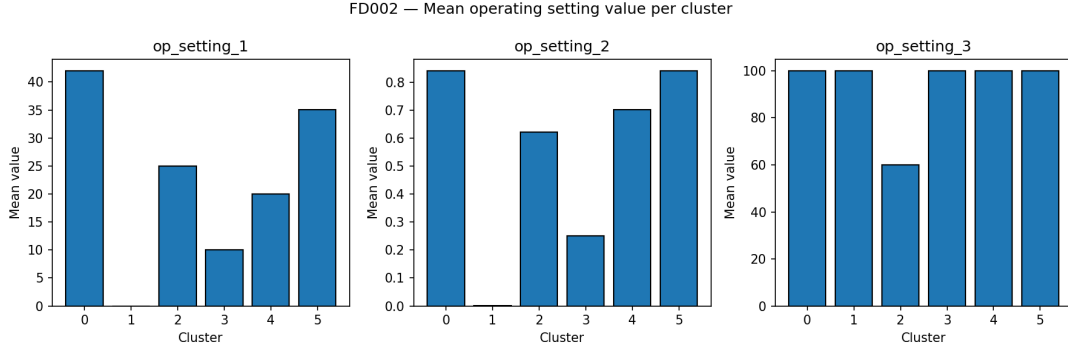


Figure 5: Mean value of each operating setting per KMeans cluster ($k = 6$) for the FD002 training set.

- Rolling standard deviation
- Linear OLS slope
- Rolling mean deviation from baseline (first window) value

To handle variable-length sequences for the feature approach the `pack_padded_sequence` and `pad_packed_sequence` functions from PyTorch are used. The MSE loss is masked to ignore the padded values.

3.3 Models

The three models are configured with the following defaults for the window approach:

- **RNN/LSTM:** two recurrent layers of hidden size 128; dropout of 0.2 between layers.
- **TCN:** four dilated causal residual blocks with channel width 64; dilations $\{1, 2, 4, 8\}$; fully connected prediction head.
- **Optimiser:** Adam, learning rate 10^{-3} .
- **Loss:** mean squared error (MSE).
- **Training:** maximum 200 epochs; early-stopping patience 10 epochs; gradient clipping with maximum norm 1.0; batch size 64.

Sequence-approach training protocol. The feature-sequence approach feeds one full engine lifetime per sample. The training pool therefore contains approximately 80 engines (FD001) or 208 engines (FD002), rather than tens of thousands of windows. With batch size 64 this yields only ≈ 2 gradient updates per epoch; combined with early-stopping patience of 10 epochs, training stops after roughly 20 gradient updates — far too few for convergence. The following sequence-specific overrides are applied: batch size 8 (≈ 10 gradient updates per epoch for FD001), learning rate 3×10^{-4} , gradient-clipping norm 0.25, early-stopping patience 50 epochs, and a maximum of 500 epochs.

Weight initialisation for long sequences. Training recurrent models on sequences of several hundred timesteps introduces two distinct instabilities.

Gradient vanishing through the recurrent weights. At each timestep, backpropagation multiplies the gradient by the hidden-to-hidden weight matrix W_{hh} . Over T timesteps the gradient is scaled by W_{hh}^T : if the singular values of W_{hh} are less than 1 the gradient vanishes; if greater than

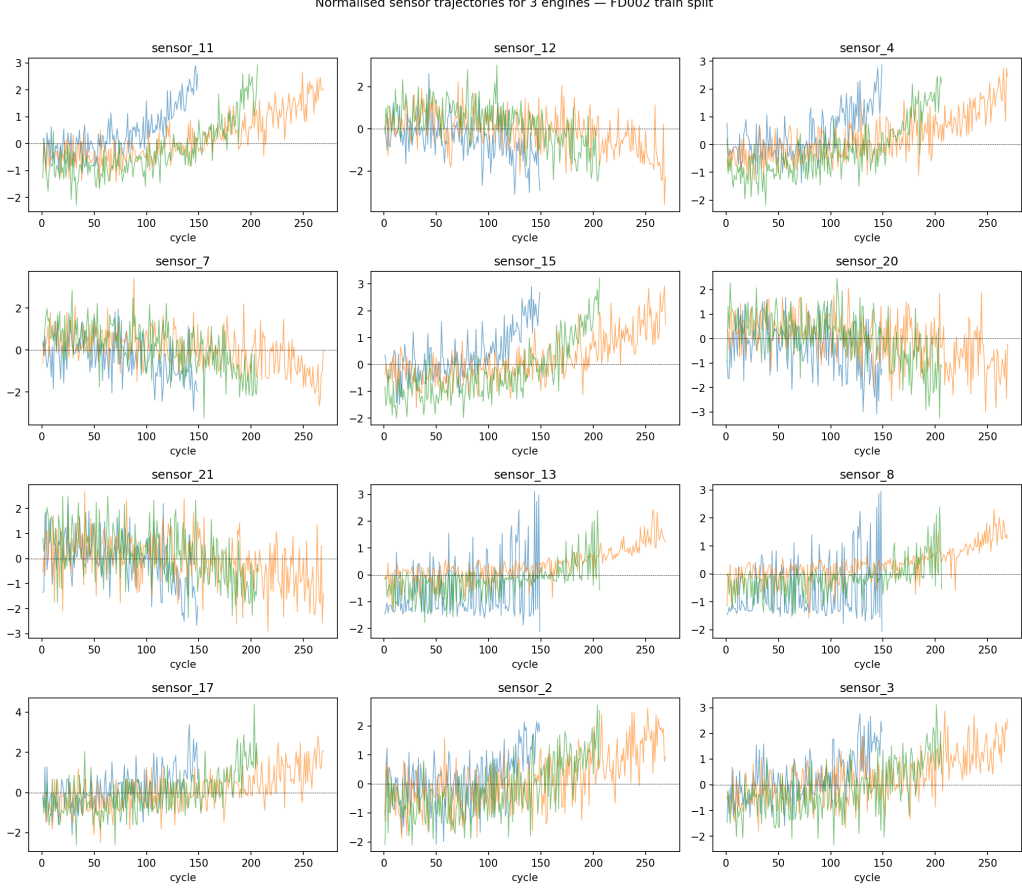


Figure 6: Per-cluster normalised sensor trajectories for a sample of FD002 engines.

1 it explodes. Both RNN and LSTM layers initialise W_{hh} orthogonally, ensuring all singular values equal exactly 1 at the start of training, so gradient magnitude is preserved across the full sequence length at initialisation.

LSTM forget gate collapse. The LSTM forget gate $f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$ controls how much of the cell state is retained at each step. With the default bias $b_f = 0$ the gate activates at $\sigma(0) = 0.5$, discarding half the cell state every timestep. Over 300 timesteps this compounds to a retention of $0.5^{300} \approx 0$, effectively destroying all long-range memory and gradient flow before training begins. The forget gate bias is therefore initialised to $b_f = 1$, raising default retention to $\sigma(1) \approx 0.73$ and allowing gradient signal to propagate across the full sequence from the first epoch.

3.4 XGBoost baseline

XGBoost is tuned with Optuna (TPE sampler, 100 trials) minimising validation RMSE. The searched hyperparameters and their ranges are listed in Table 1.

4 Results

4.1 Metrics

Two metrics are reported. RMSE measures average prediction error symmetrically. The NASA score is an asymmetric exponential penalty that penalises late predictions (overestimating RUL) more heavily than early ones, reflecting the higher operational cost of missing an impending failure. NASA score is not used as a training loss. Five fixed random seeds (42, 123, 456,

Parameter	Range	Scale	Type
learning_rate	[0.001, 0.3]	log	continuous
max_depth	[3, 10]	uniform	integer
subsample	[0.5, 1.0]	uniform	continuous
colsample_bytree	[0.5, 1.0]	uniform	continuous
min_child_weight	[1, 20]	uniform	integer
reg_lambda	[0.01, 10.0]	log	continuous

Table 1: Optuna hyperparameter search space for XGBoost. `n_estimators` is fixed at 1000 and controlled by early stopping (patience = 30 rounds).

789, 1011) are used for model initialisation to probe sensitivity to initial conditions; mean and standard deviation are reported across seeds.

4.2 FD001

Results are shown in Table 2.

Approach	Model	RMSE (mean $\pm$ std)	NASA (mean $\pm$ std)	NASA/engine
window	RNN	15.82 $\pm$ 0.57	502 $\pm$ 100	5.0
window	LSTM	14.88 $\pm$ 0.36	444 $\pm$ 65	4.4
window	TCN	16.55 $\pm$ 0.32	477 $\pm$ 37	4.8
sequence	RNN	20.66 $\pm$ 3.72	646 $\pm$ 171	6.5
sequence	LSTM	35.65 $\pm$ 10.80*	13 881 $\pm$ 7 554*	139*
sequence	TCN	17.60 $\pm$ 0.24	508 $\pm$ 24	5.1
sequence	XGBoost	14.46 $\pm$ 0.05	386 $\pm$ 15	3.9

Table 2: Mean and standard deviation of RMSE and NASA score across five seeds per model and data approach on FD001 test set ($N = 100$ test engines). NASA/engine is the mean NASA total divided by N and enables cross-dataset comparison. *Sequence LSTM results are bimodal: one of five seeds converges to ≈ 14 RMSE while four collapse to constant-output predictors (≈ 40 RMSE); the mean and standard deviation therefore do not represent a single underlying performance level.

4.3 FD002

The same five seeds and model configurations as in the FD001 evaluation are used. Results are shown in Table 3.

Approach	Model	RMSE (mean $\pm$ std)	NASA (mean $\pm$ std)	NASA/engine
window	RNN	15.44 $\pm$ 0.86	1120 $\pm$ 118	4.3
window	LSTM	14.78 $\pm$ 0.60	1132 $\pm$ 169	4.4
window	TCN	16.45 $\pm$ 0.42	1217 $\pm$ 117	4.7
sequence	RNN	14.59 $\pm$ 0.71	834 $\pm$ 62	3.2
sequence	LSTM	37.18 $\pm$ 13.43*	41 344 $\pm$ 22 688*	160*
sequence	TCN	16.73 $\pm$ 0.53	1104 $\pm$ 40	4.3
sequence	XGBoost	14.10 $\pm$ 0.01	775 $\pm$ 7	3.0

Table 3: Mean and standard deviation of RMSE and NASA score across five seeds per model and data approach on FD002 test set ($N = 259$ test engines). NASA/engine is the mean NASA total divided by N and enables cross-dataset comparison. *Sequence LSTM results are bimodal: one of five seeds converges while four collapse to constant-output predictors; see Table 2 footnote.

4.4 Fair comparison

The window approach and XGBoost baseline use the suggested configuration without modification. The feature-sequence approach deviates from the defaults on batch size, learning rate, gradient clipping, early-stopping patience, epoch budget, and weight initialisation, as described in Section 3.3. These deviations are confined to the sequence approach and are motivated by the substantially smaller number of gradient updates per epoch that arises from engine-level batching; they do not affect the window approach or XGBoost and are applied identically across all sequence models and both datasets.

5 Focused hyperparameter study

5.1 Setup

LSTM is selected as the architecture for the hyperparameter study, as it achieves the best mean RMSE among the three deep learning architectures in the window approach on FD001. The study covers both the window and feature-sequence approaches on both FD001 and FD002, and searches over three axes:

- **Window size** $w \in \{20, 30, 50\}$ — controls the temporal context available to the model per prediction.
- **Hidden size** $h \in \{64, 128\}$ — controls model capacity.
- **Learning rate** $\eta \in \{10^{-4}, 10^{-3}\}$ — controls optimisation step size.

All other hyperparameters are fixed at the defaults described in Section 3.3. Each of the $3 \times 2 \times 2 = 12$ configurations per approach is trained with 3 fixed seeds (42, 123, 456), giving 72 runs per dataset. Validation RMSE averaged across seeds is used to rank configurations; the best configuration per approach is subsequently evaluated on the test set with the same 3 seeds.

One methodological limitation applies to the sequence approach: the training loop uses a dedicated `sequence_lr` override (3×10^{-4}) that takes precedence over `cfg["training"]["lr"]`, so the learning rate axis has no effect on sequence runs. The sequence grid therefore reduces to $3 \times 2 = 6$ distinct configurations ($w \times h$), with the η dimension being redundant.

5.2 Analysis

Window approach — FD001. Table 4 reports validation RMSE for all 12 window-approach configurations on FD001. The best configuration is $w=50, h=128, \eta=10^{-3}$ with a mean validation RMSE of 12.15 ± 0.11 , achieving a test RMSE of 15.21 ± 0.49 across the three evaluation seeds.

Three clear patterns emerge. *Window size is the dominant axis:* $w=50$ outperforms $w=30$ which outperforms $w=20$ at every combination of h and η , with an improvement of approximately 2.5 RMSE points from the smallest to the largest window. This indicates that longer context consistently helps the model and that the sensor degradation signal is not fully captured within 20 or 30 cycles. *Hidden size has a smaller but consistent effect:* $h=128$ beats $h=64$ at every (w, η) pair, suggesting the model benefits from additional capacity, but the gap (≈ 0.5 RMSE) is small relative to the window size effect. *Learning rate:* $\eta=10^{-3}$ is consistently better than $\eta=10^{-4}$, indicating the default learning rate is well suited to this task and that a slower rate undertrains within the allowed epoch budget.

Grid-edge check: the best configuration lies at the boundary of the search grid on window size ($w=50$). This suggests that further gains may be available with larger windows, and a follow-up search extending to $w \in \{70, 100\}$ would be warranted.

w	h	η	Val RMSE (mean $\pm$ std)
50	128	10^{-3}	12.15 ± 0.11
50	64	10^{-3}	12.64 ± 0.10
30	128	10^{-3}	12.96 ± 0.21
30	64	10^{-3}	13.42 ± 0.34
50	64	10^{-4}	13.94 ± 0.97
50	128	10^{-4}	14.01 ± 0.38
30	128	10^{-4}	14.15 ± 0.19
30	64	10^{-4}	14.22 ± 0.38
20	128	10^{-3}	14.59 ± 0.18
20	128	10^{-4}	15.41 ± 0.25
20	64	10^{-3}	15.58 ± 0.22
20	64	10^{-4}	15.74 ± 0.49

Table 4: Window-approach LSTM hyperparameter grid on FD001, sorted by mean validation RMSE across 3 seeds.

Default configuration retrospective: the default ($w=30$, $h=128$, $\eta=10^{-3}$) ranks third overall with a validation RMSE of 12.96. It was a reasonable starting point — the hidden size and learning rate choices were optimal — but using $w=50$ would have improved results by approximately 0.8 RMSE points with no other changes.

Window approach — FD002. Table 5 reports validation RMSE for the FD002 window grid. The best configuration is again $w=50$, $h=128$, $\eta=10^{-3}$ with a mean validation RMSE of 14.08 ± 0.26 , achieving a test RMSE of 13.62 ± 0.86 .

w	h	η	Val RMSE (mean $\pm$ std)
50	128	10^{-3}	14.08 ± 0.26
50	64	10^{-3}	14.69 ± 0.18
30	128	10^{-3}	15.51 ± 0.25
50	64	10^{-4}	15.51 ± 0.13
50	128	10^{-4}	16.22 ± 0.10
30	64	10^{-3}	16.30 ± 0.20
30	64	10^{-4}	16.69 ± 0.42
30	128	10^{-4}	17.21 ± 0.44
20	128	10^{-3}	18.02 ± 0.46
20	64	10^{-3}	18.56 ± 0.12
20	64	10^{-4}	18.78 ± 0.35
20	128	10^{-4}	18.91 ± 0.16

Table 5: Window-approach LSTM hyperparameter grid on FD002, sorted by mean validation RMSE across 3 seeds.

The same ranking of axes holds on FD002: window size dominates, hidden size has a smaller consistent effect, and $\eta=10^{-3}$ outperforms $\eta=10^{-4}$. The best configuration is identical to FD001 ($w=50$, $h=128$, $\eta=10^{-3}$), confirming that the optimal window-approach hyperparameters generalise across operating conditions.

Feature-sequence approach. Because the learning rate axis is ineffective for the sequence approach (see Setup), only the $w \times h$ sub-grid of 6 configurations is informative. Table 6 summarises the 6 distinct sequence configurations on both datasets.

Two findings stand out. *Smaller windows are better for sequence LSTM*, in direct contrast

w	h	Val RMSE (mean $\pm$ std)	
		FD001	FD002
20	128	13.34 $\pm$ 1.80	11.81 $\pm$ 0.13
20	64	23.06 $\pm$ 0.44	13.86 $\pm$ 0.82
30	64	22.82 $\pm$ 0.56	13.68 $\pm$ 0.49
30	128	33.01 $\pm$ 15.42	32.00 $\pm$ 16.95
50	64	30.12 $\pm$ 9.80	32.66 $\pm$ 14.65
50	128	41.44 $\pm$ 0.00	31.81 $\pm$ 16.11

Table 6: Sequence-approach LSTM hyperparameter grid (6 distinct configurations, sorted by FD001 val RMSE). High means reflect bimodal instability: some seeds converge while others collapse to constant predictors. Bold marks the best configuration.

to the window approach where $w=50$ dominated. With $w=20$, each engine sequence has fewer timesteps, reducing the risk of vanishing gradients during backpropagation through time. Larger windows ($w=50$) produce near-complete collapse on FD001 and high instability on FD002.

FD002 is substantially more stable than FD001 under the sequence approach. The best configuration ($w=20$, $h=128$) achieves a validation RMSE of 11.81 ± 0.13 on FD002 — low variance, all three seeds converging — and a test RMSE of 12.59 ± 0.68 . On FD001 the same configuration is more erratic (val RMSE 13.34 ± 1.80 , test RMSE 15.85 ± 0.80). The larger FD002 training set (≈ 208 engines vs ≈ 80) provides a stronger gradient signal that keeps more seeds from collapsing.

6 Comparative Analysis

6.1 Within-approach comparison

Raw window approach. Across both datasets, LSTM achieves the lowest mean RMSE (FD001: 14.88 ± 0.36 ; FD002: 14.78 ± 0.60), followed by RNN (15.82 ± 0.57 ; 15.44 ± 0.86) and TCN (16.55 ± 0.32 ; 16.45 ± 0.42). The margins between architectures are modest (approximately 2 RMSE points) and the LSTM–RNN gap lies within one standard deviation of both models, indicating that all three architectures are broadly comparable under the window paradigm.

Feature-sequence approach. TCN is the most consistent architecture on both datasets (FD001: 17.60 ± 0.24 ; FD002: 16.73 ± 0.53), with low variance across seeds, consistent with its dilated convolutional structure being insensitive to sequence length. After applying sequence-specific training fixes (reduced learning rate, tighter gradient clipping, orthogonal hidden-weight initialisation, and LSTM forget-gate bias initialisation), RNN converges reliably across all seeds on both datasets (FD001: 20.66 ± 3.72 ; FD002: 14.59 ± 0.71). Sequence LSTM, by contrast, remains highly unstable: only one of five seeds converges on each dataset, while the remaining four collapse to near-constant predictors (≈ 40 RMSE). Multiple stabilisation measures were applied — including lower learning rate, gradient clipping, orthogonal weight initialisation, and forget-gate bias initialisation to 1 — yet the collapse persists for most seeds. This suggests that the combination of long padded sequences, a masked sequence-to-sequence loss, and small batch sizes creates an optimisation landscape where LSTM is more susceptible to getting stuck than vanilla RNN, despite LSTM’s gating mechanism being specifically designed for long-range dependencies. Counterintuitively, RNN is more reliably trainable than LSTM under the sequence paradigm.

6.2 Between-approach comparison

RNN. On FD001, the window approach outperforms the sequence approach (15.82 vs 20.66 RMSE). On FD002, however, the relationship reverses: sequence RNN (14.59 ± 0.71) outperforms window RNN (15.44 ± 0.86), making it the only configuration where the sequence approach strictly beats the window approach for a recurrent model. The likely explanation is that FD002 provides substantially more training engines (≈ 208 vs ≈ 80 for FD001), giving the sequence model better generalisation despite the longer sequences. The earlier pre-fix collapse (≈ 44 RMSE) was confirmed to be a training problem rather than a capacity limitation: with appropriate initialisations and hyperparameters, sequence RNN is competitive with the best models overall.

LSTM. The window approach leads on both datasets (FD001: 14.88 vs 35.65; FD002: 14.78 vs 37.18 RMSE), but these comparisons are heavily distorted by the bimodal instability. The single seed that converges on each dataset achieves ≈ 14 RMSE — competitive with window LSTM — demonstrating that the sequence LSTM has the capacity to perform well. The large between-approach gap is entirely a training reliability problem, not a capacity ceiling.

TCN. The two approaches are nearly equivalent for TCN (FD001: 16.55 vs 17.60; FD002: 16.45 vs 16.73). TCN’s local dilated convolutions aggregate information over a bounded receptive field rather than backpropagating through the full sequence, making it robust to sequence length and paradigm choice alike.

6.3 Deep learning vs. XGBoost

XGBoost is the best-performing model overall (FD001: 14.46 ± 0.05 RMSE, 386 ± 15 NASA score; FD002: 14.10 ± 0.01 RMSE, 775 ± 7 NASA score), outperforming the best deep learning configuration on both metrics and both datasets. The RMSE margin over window LSTM (FD001: 14.88; FD002: 14.78) is small and falls within seed variability, but the NASA score advantage is more consistent.

XGBoost receives a single fixed-size window per prediction and treats each window as an independent sample. Its competitive performance suggests that the handcrafted window statistics already summarise the relevant degradation signal, and that the additional temporal context provided by processing longer histories or full sequences is not being translated into a meaningful predictive advantage by the deep learning models tested here. This is a finding about the data rather than a verdict on deep learning in general: the CMAPSS sensor trajectories may degrade in a sufficiently regular and Markovian fashion that a short local window is sufficient.

6.4 FD001 vs. FD002

RMSE. Window model RMSE is stable across datasets (RNN: $15.82 \rightarrow 15.44$; LSTM: $14.88 \rightarrow 14.78$; TCN: $16.55 \rightarrow 16.45$), indicating that the per-cluster z-score normalisation effectively absorbs the additional operating-condition variability of FD002. Sequence TCN shows a small improvement on FD002 ($17.60 \rightarrow 16.73$), consistent with the appended `op_condition` cluster label providing a useful conditioning signal under multi-condition data.

NASA score. The raw NASA totals are roughly 2.5 times higher on FD002 than FD001 for all models (e.g. window LSTM: $444 \rightarrow 1\,132$; XGBoost: $386 \rightarrow 775$). This is largely a dataset-size effect: FD002 has 259 test engines versus 100 for FD001, and the NASA score is a sum over engines, so the total scales with N even at identical per-engine accuracy. The NASA/engine column in the results tables corrects for this: per-engine NASA scores are almost identical across datasets for window models (e.g. window LSTM: 4.4 on both FD001 and FD002), confirming that the per-cluster normalisation keeps prediction quality stable across operating conditions.

Sequence RNN and LSTM on FD002. Sequence RNN converges reliably on FD002 (14.59 ± 0.71 RMSE), outperforming sequence TCN (16.73), window RNN (15.44), and matching XGBoost (14.10) — making it the best deep learning model on FD002 overall. Sequence LSTM on FD002 remains unstable: one of five seeds converges while four collapse, and the mean RMSE of 37.18 ± 13.43 is not representative. The forget-gate bias initialisation ($b_f = 1$) moved FD002 LSTM from zero converging seeds to one, but full stabilisation was not achieved. The contrast between reliable sequence RNN and unreliable sequence LSTM on the same data points to LSTM’s gate interactions — rather than sequence length or data complexity — as the primary source of instability under this training regime.

6.5 RMSE vs. NASA score

Rankings by RMSE and NASA score agree in most cases: configurations that lead on RMSE also tend to lead on NASA score. The clearest exception is sequence LSTM, where bimodal collapse produces extreme NASA scores on both datasets (FD001: 13 881; FD002: 41 344) despite RMSE values of 35.65 and 37.18 respectively. The collapsed seeds predict a near-constant value of approximately 79 cycles for every test engine, systematically overestimating RUL; the exponential NASA penalty then amplifies each overestimation into a very large score, and the effect compounds across all test engines.

The FD002 NASA inflation for otherwise well-performing models highlights a limitation of using RMSE as the sole selection criterion. A model may achieve acceptable RMSE while carrying a systematic bias toward late predictions that is operationally costly in contexts where overestimating remaining useful life has safety consequences. Jointly monitoring RMSE and NASA score exposes this asymmetry in a way that RMSE alone does not.

6.6 Revised picture from the hyperparameter study

The main results above use the default window size $w=30$ for all models and approaches. The hyperparameter study (Section 5) reveals that this choice is particularly unfavourable for sequence LSTM: the bimodal instability and high mean RMSE observed in Tables 2 and 3 are largely a consequence of $w=30$ being too large for stable sequence training.

With $w=20$ and $h=128$, sequence LSTM converges reliably on FD002 (test RMSE 12.59 ± 0.68 across 3 seeds), outperforming window LSTM (14.78 ± 0.60) by more than 2 RMSE points. On FD001 the same configuration is less stable (15.85 ± 0.80) but still competitive with the window approach. The conclusion that sequence LSTM is unreliable should therefore be qualified: it is unreliable at $w=30$, but the hyperparameter study shows a configuration exists where it is both stable and the best-performing model on FD002.

The between-approach gap for LSTM is thus partly a hyperparameter artefact rather than an inherent architectural limitation. For RNN and TCN the hparam study does not change the conclusions: sequence RNN was already competitive at the default $w=30$, and sequence TCN’s insensitivity to sequence length makes the window size effect smaller.